

Dynamic Metadata-Driven LWC Framework

Complete Technical & Configuration Reference

Version: 2.0 | **Status:** Production Ready | **Last Updated:** 2025

Audience: Salesforce Admins (Sections 1-8, 11) · Developers (All Sections)

Table of Contents

1. [Overview](#)
2. [Architecture](#)
3. [Component Inventory](#)
4. [Custom Metadata Reference](#)
 - 4.1 [Form Config \(\(Form Config mdt\)\)](#)
 - 4.2 [Form Section \(\(Form Section mdt\)\)](#)
 - 4.3 [Form Field \(\(Form Field mdt\)\)](#)
5. [JSON Configuration Reference](#)
 - 5.1 [Visibility Logic](#)
 - 5.2 [Required Logic](#)
 - 5.3 [Formula Logic](#)
 - 5.4 [Dynamic SOQL](#)
 - 5.5 [Button Config JSON](#)
 - 5.6 [Matrix Section Config JSON](#)
6. [LWC Component Reference](#)
7. [Apex API Reference](#)
8. [Admin Configuration Guide](#)
9. [Developer Guide](#)
10. [Deployment & Integration Guide](#)
11. [Real-World Form Examples](#)
12. [Security Model](#)
13. [Performance & Governor Limits](#)
14. [Known Limitations & Workarounds](#)
15. [Troubleshooting](#)
16. [Framework vs. OmniStudio](#)

1. Overview

The **Dynamic Metadata-Driven LWC Framework** is a native Salesforce solution that generates fully functional, multi-object record intake forms entirely from Custom Metadata Type (CMDT) configuration — **no new LWC code required per form**.

What problem does it solve?

Standard Salesforce Dynamic Forms and Page Layouts cannot:

- Create a **parent record and multiple child records** in a single, atomic transaction
- Apply **field-level conditional visibility** evaluated entirely on the client without server round-trips
- React in real time to field changes using **server-side SOQL queries** as dependencies
- Handle **2D matrix data entry** tied to junction objects

Building bespoke LWC components for each new form request creates high technical debt. This framework solves all of the above through configuration, not code.

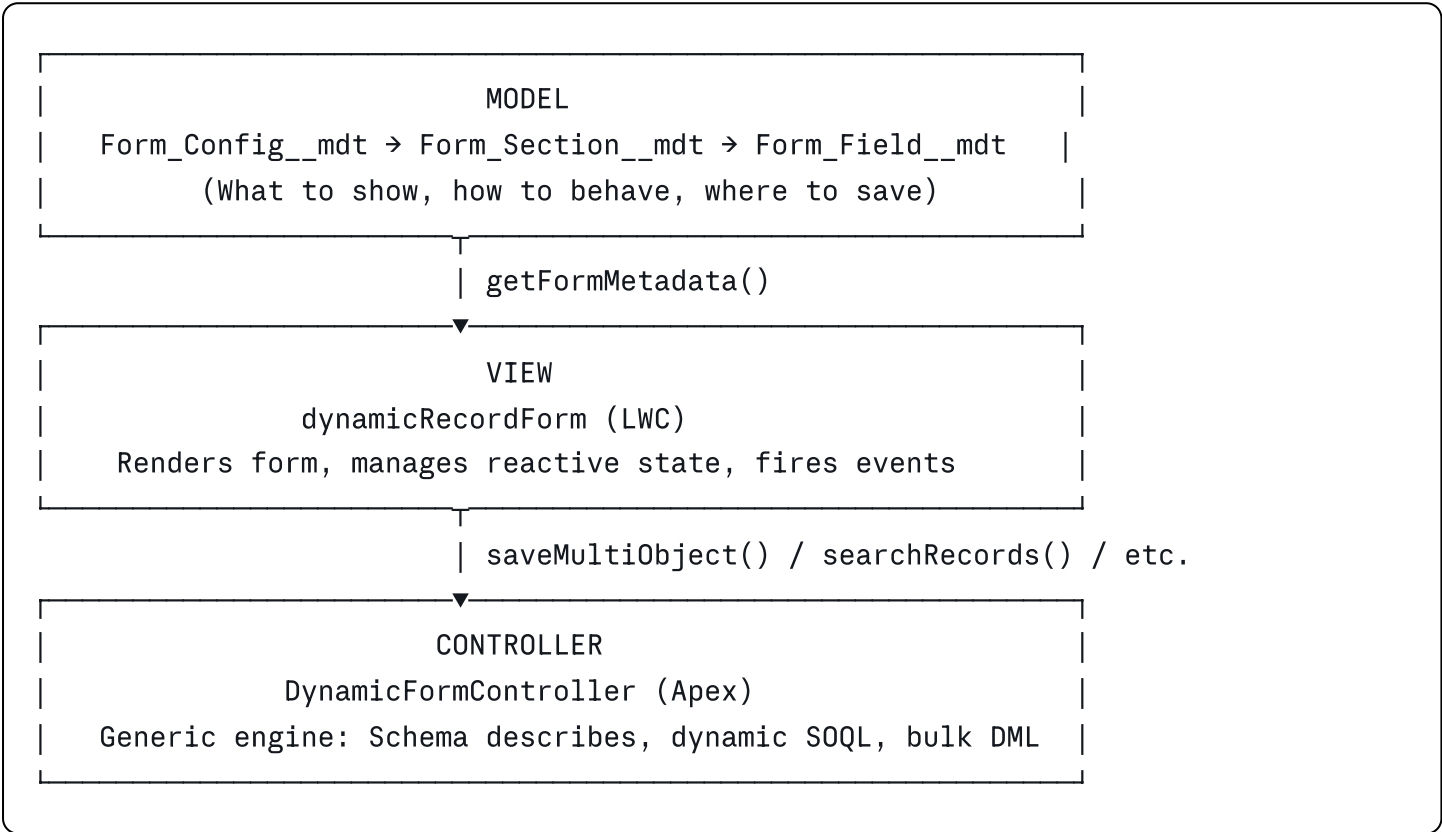
What does it deliver?

Capability	Details
Multi-object transactional save	Parent + N child objects in one atomic database operation
Metadata-driven UI	Sections, fields, columns, and types all configured via CMDT
Client-side visibility engine	AND/OR conditions evaluated instantly on field change
Dependent picklists	Standard schema dependencies + custom JSON overrides
Record Type-aware picklists	Filters picklist values based on active Record Type via ConnectAPI (Spring '26+)
Formula fields	Cross-section JS math/string expressions, results excluded from DML
Dynamic SOQL fields	Real-time field values fetched from the database based on other field inputs
2D Matrix grids	Configurable column/row grids backed by a junction object
File uploads	Supports both <code>lightning-file-upload</code> (standard) and base64 upload (Lightning Out/VF)

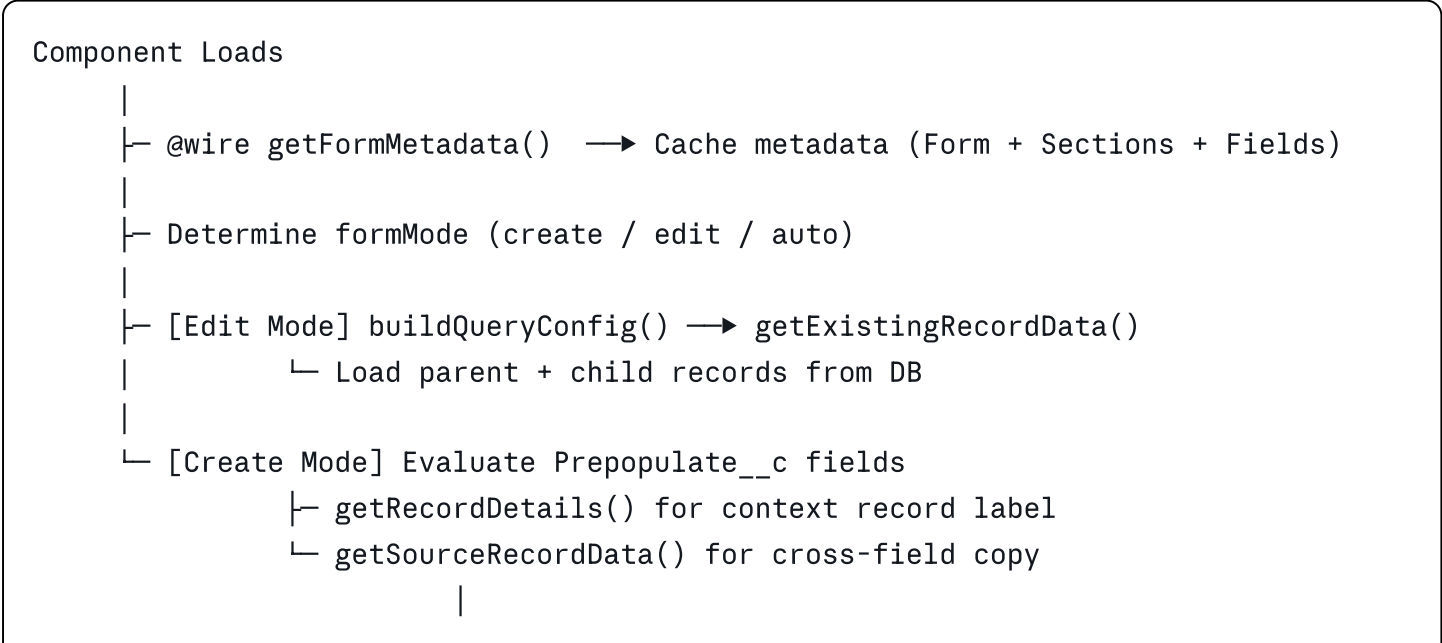
Capability	Details
Wizard mode	Multi-step form with Previous/Next navigation
Context-aware pre-population	Auto-fills lookups based on the record context the form was launched from

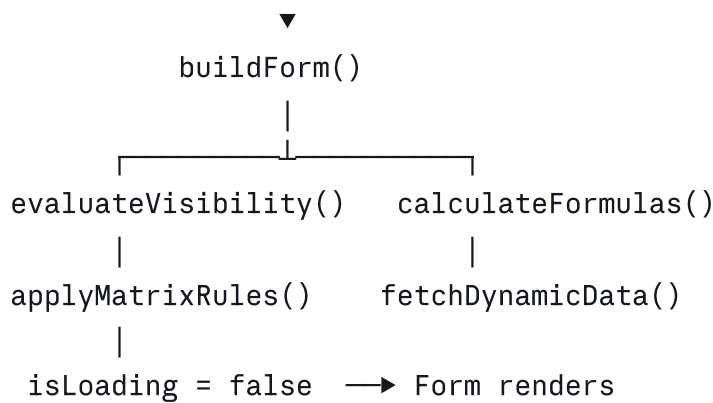
2. Architecture

2.1 MVC Pattern



2.2 Initialization Sequence





2.3 Save Sequence

User clicks Submit

- |
- |─ `validateCurrentStep()` – `reportValidity()` on all visible inputs
 - | └ Auto-expands collapsed sections with errors
- |
- |─ Build hierarchical payload from flat `sectionData` state
 - | └ Strips: non-visible fields, `Save_To_Database__c=false` fields
- |
- |─ `saveMultiObject()` [Apex]
 - | └ `Database.setSavepoint()`
 - | └ DELETE orphaned records (`_recordsToDelete` list)
 - | └ UPSERT parent record
 - | └ UPSERT child records (injects parent ID via `relationshipMap`)
 - | └ ROLLBACK on any failure
- |
- |─ [Phase 2, if Matrix has parent section dependency]
 - | └ `saveMultiObject()` again with resolved child IDs
- |
- |─ Success → `notifyRecordUpdateAvailable()` → fire 'close' event

2.4 Key Design Decisions

Decision	Reasoning
Client-side visibility evaluation	Zero server round-trips on field change — instant UI response
<code>_lastLoadKey</code> deduplication guard	Prevents double initialization when both <code>recordId</code> and <code>objectApiName</code> setters fire on the same render cycle
<code>Key_Prefix__c</code> for pre-population	Prevents polymorphic ID mismatch — e.g., a Contact ID (<code>003...</code>) must never populate an Account lookup field

Decision	Reasoning
<code>SystemModeDML</code> inner class	Allows <code>saveMultiObject</code> to bypass sharing rules when <code>Save_Without_Sharing__c = true</code> , while keeping the outer class <code>with sharing</code> compliant
<code>@AuraEnabled(cacheable=true)</code> on metadata	Metadata rarely changes; client-side caching makes repeat form loads near-instant
Linear promise chain during init	Prevents the race condition where the form renders before lookup labels are fetched for Edit Mode

3. Component Inventory

Salesforce Components

Component	Type	Purpose
<code>dynamicRecordForm</code>	LWC	Core rendering engine. Receives a <code>formDeveloperName</code> and renders the entire form
<code>caseCreateWrapper</code>	LWC	Wrapper for creating Cases. Handles navigation (Quick Action + Lightning Out/VF)
<code>caseEditWrapper</code>	LWC	Wrapper for editing Cases via Quick Action
<code>taskCreateWrapper</code>	LWC	Wrapper for creating Tasks via Quick Action from a Case
<code>calculateQcScore</code>	LWC	Wrapper for the QC Score edit form on Cases
<code>DynamicFormApp</code>	Aura Application	Lightning Out host for the VF page integration
<code>MaintenanceFormAuraWrapper</code>	Aura Component	Tab-based wrapper for Maintenance form in Console apps
<code>DynamicFormController</code>	Apex Class	All server-side operations (metadata fetch, data load, save, search)
<code>PicklistDependencyHelper</code>	Apex Class	Decodes Salesforce's internal <code>validFor</code> base64 bitset for picklist dependencies
<code>Form_Config__mdt</code>	Custom Metadata	Root form configuration

Component	Type	Purpose
<code>Form_Section__mdt</code>	Custom Metadata	Section-level configuration
<code>Form_Field__mdt</code>	Custom Metadata	Field-level configuration
<code>Matrix_Data__c</code>	Custom Object	Junction object storing matrix grid cell values
<code>caseCreatePage</code>	Visualforce Page	Entry point that routes to custom LWC or standard new Case form

Quick Actions

Action API Name	Object	Component	Purpose
<code>Account.Create_New_Case</code>	Account	<code>caseCreateWrapper</code>	Create Case from Account
<code>Case.Create_Child_Case</code>	Case	<code>caseCreateWrapper</code>	Create child Case from Case
<code>Case.Create_Maintenance_Task</code>	Case	<code>taskCreateWrapper</code>	Create Task from Case
<code>Case.Edit_Case_Details</code>	Case	<code>caseEditWrapper</code>	Edit Case via dynamic form
<code>Case.QC_Score</code>	Case	<code>calculateQcScore</code>	Update QC Score on Case

4. Custom Metadata Reference

4.1 Form Config (`Form_Config__mdt`)

The root record that defines one complete form. Every form has exactly one `Form_Config__mdt` record.

Field Label	API Name	Type	Required	Description
Label	<code>MasterLabel</code>	Text	✓	Human-readable name shown in Setup
Developer Name	<code>DeveloperName</code>	Text	✓	The value passed to <code>formDeveloperName</code> on the LWC. Must be unique. Use underscores, no spaces
Active	<code>Active__c</code>	Checkbox	✓	Master on/off switch. If <code>false</code> , Apex throws a user-friendly error and the form refuses to load
Object API Name	<code>Object_API_Name__c</code>	Text	✓	The API name of the primary/parent Salesforce object (e.g., <code>Case</code> , <code>Work_Order__c</code>)
Form Title	<code>Form_Title__c</code>	Text	—	Text shown in the Lightning Card header. Defaults to <code>"Dynamic Form"</code> if blank
Form Icon	<code>Form_Icon__c</code>	Text	—	SLDS icon name for the card header (e.g., <code>standard:case</code> , <code>standard:account</code>). Defaults to <code>standard:record</code>
Display Mode	<code>Display_Mode__c</code>	Picklist	—	<code>Single Page</code> (all sections visible at once) or <code>Wizard</code> (step-through with Next/Previous buttons). Defaults to <code>Single Page</code>
Form Instructions	<code>Form_Instructions__c</code>	Long Text	—	HTML-formatted banner displayed at the top of the form (rendered via <code>lightning-formatted-rich-text</code>)
Button Config JSON	<code>Button_Config_JSON__c</code>	Long Text	—	JSON to override default button labels and toast messages. See Section 5.5

Field Label	API Name	Type	Required	Description
Save Without Sharing	Save_Without_Sharing__c	Checkbox	—	If <code>true</code> , DML operations bypass sharing rules. Use when Case Assignment Rules immediately reassign record ownership after insert , which would otherwise cause a cross-reference DML error

4.2 Form Section ((Form_Section__mdt))

Each section represents a collapsible group of fields within a form. Sections can target the parent object or a child object.

Field Label	API Name	Type	Required	Description
Label	<code>MasterLabel</code>	Text	✓	Default section header characters
Developer Name	<code>DeveloperName</code>	Text	✓	Unique identifier used internally and in Matrix/section dependencies
Form Config	<code>Form_Config__c</code>	Metadata Relationship	✓	Links to the parent <code>Form_Config__mdt</code>
Object API Name	<code>Object_API_Name__c</code>	Text	—	The Salesforce object section's fields belong to. If different from the parent Config object, this section is treated as a child section
Order	<code>Order__c</code>	Number	—	Display order (ascending). Sections without a value appear last
Override Label	<code>Override_Label__c</code>	Text (255)	—	Overrides <code>MasterLabel</code> on the rendered form. Up to 255 characters
Render As	<code>Render_As__c</code>	Picklist	—	<code>Standard</code> (default) or <code>Matrix</code> (2D grid) Section 5.6
Number of Columns	<code>Number_of_Columns__c</code>	Number	—	<code>1</code> or <code>2</code> columns in a grid. Defaults to <code>2</code> . Warning: (Long Text, File Upload, Select) always span full width regardless of this setting
Allow Multiple Rows	<code>Allow_Multiple_Rows__c</code>	Checkbox	—	Adds Add and Delete buttons to the section. Each added row creates one record on save. Use for objects where multiple records are expected (e.g., Work Items)
Relationship Parent Field	<code>Relationship_Parent_Field__c</code>	Text	—	Required if <code>Object_API_Name__c</code> is set

Field Label	API Name	Type	Required	Description
				child object. The API uses the lookup/master-detail relationship on the child object to get the value from the parent (e.g., <code>Maintenance_Request__c</code>)
Is Required	<code>Is_Required__c</code>	Checkbox	—	For Matrix sections of a Matrix section, if <code>true</code> , the user must select at least one cell before saving the record.
Collapse by Default	<code>Collapse_by_Default__c</code>	Checkbox	—	If <code>true</code> , the section is collapsed on load. User can click to expand.
Show On	<code>Show_On__c</code>	Picklist	—	<code>Create</code> (only in create mode), <code>Edit</code> (only in edit mode), <code>Both</code> (always visible), <code>Blank</code> (blank, defaults to <code>Blank</code>)
Custom Component Name	<code>Custom_Component_Name__c</code>	Text	—	Developer-only. If populated, renders a custom child component instead of a field grid. The component must implement the same contract as the <code>FieldGrid</code> framework.
Visibility Logic	<code>Visibility_Logic__c</code>	Long Text	—	JSON condition to show or hide the entire section. See Section 5.1 .
Section Config JSON	<code>Section_Config_JSON__c</code>	Long Text	—	Required when <code>Render_As__c = Table</code> . Defines columns, rows, and per-cell rules. See Section 5.2 .
Parent Section Developer Name	<code>Parent_Section_DeveloperName__c</code>	Text	—	For Matrix sections of a Matrix section's data to a different section object (rather than the main parent), enter the section's <code>DeveloperName</code> here. Enables a two-way relationship between the parent and child sections.

4.3 Form Field (`Form_Field__mdt`)

Defines a single input, display element, or virtual calculation field within a section.

Core Identity

Field Label	API Name	Type	Required	Description
Label	<code>MasterLabel</code>	Text		Default field label. Max 40 characters
Developer Name	<code>DeveloperName</code>	Text		Unique system identifier
Form Section	<code>Form_Section__c</code>	Metadata Relationship		Links to the parent <code>Form_Section__mdt</code> record
Field API Name	<code>Field_API_Name__c</code>	Text	—	The exact Salesforce field API name on the target object. If blank, the field is treated as a Form-Only virtual field and never saved to the database
Field Type	<code>Field_Type__c</code>	Picklist		Controls the rendered UI component. See field type table below
Order	<code>Order__c</code>	Number	—	Display order within the section (ascending)
Override Label	<code>Override_Label__c</code>	Text (200)	—	Overrides <code>MasterLabel</code> on the form. Use for labels exceeding 40 characters

Field Types

Field_Type__c Value	Renders As	Notes
Text	<code>lightning-input</code> <code>type="text"</code>	Standard text input
Number	<code>lightning-input</code> <code>type="number"</code>	Numeric input with <code>step="any"</code>
Checkbox	<code>lightning-input</code> <code>type="checkbox"</code>	Boolean toggle
Date	<code>lightning-input</code> <code>type="date"</code>	Date picker
DateTime	<code>lightning-input</code> <code>type="datetime"</code>	Date+time picker
Picklist	<code>lightning-combobox</code>	Single-select dropdown with <code>--None--</code> option prepended
Multi-Select Picklist	<code>lightning-dual-</code> <code>listbox</code>	Dual listbox; values stored semicolon-delimited
Lookup	Custom combobox with search	Custom LWC lookup with live search. Requires <code>Lookup_Target_Object__c</code> and <code>Lookup_Search_Field__c</code>
Long Text Area	<code>lightning-textarea</code>	Multi-line text; always spans full width
File Upload	<code>lightning-file-upload</code> or <code>lightning-input</code> <code>type="file"</code>	Standard context uses <code>lightning-file-upload</code> ; Lightning Out/VF uses base64 upload. See Section 9.3
Header	Styled <code><h3></code> divider	Non-input decorative element. Never saved
Display Text	<code>lightning-formatted-</code> <code>rich-text</code>	Renders <code>HTML_Content__c</code> as rich text. Never saved

Validation & Behaviour

Field Label	API Name	Type	Description
Required	<code>Required__c</code>	Checkbox	Statically marks the field as required. Enforced on both client and server
Required Logic	<code>Required_Logic__c</code>	Long Text	JSON condition to make the field conditionally required. Evaluated in real time. See Section 5.2
Read Only	<code>Read_Only__c</code>	Checkbox	Renders the field as disabled. Note: Fields with a <code>Formula_Logic__c</code> value are automatically read-only regardless of this setting
Save To Database	<code>Save_To_Database__c</code>	Checkbox	Defaults to <code>true</code> . Set to <code>false</code> for virtual/calculation-only fields. These fields are completely stripped from the DML payload
Fetch On Edit	<code>Fetch_On_Edit__c</code>	Checkbox	If <code>true</code> , this field's value is fetched from the database even when its parent section is hidden in edit mode. Use for fields that drive visibility logic but are not directly displayed
Default Value	<code>Default_Value__c</code>	Text	Initial value populated on create mode. For Checkbox fields, use <code>true</code> or <code>false</code> . For Number fields, use numeric string (e.g., <code>0</code>)
Help Text	<code>Help_Text__c</code>	Text	Displayed below the field in small italic text
HTML Content	<code>HTML_Content__c</code>	Long Text	Only used when <code>Field_Type__c = Display Text</code>. Supports HTML tags (<code></code> , <code>
</code> , <code><a></code> , etc.)
Visibility Logic	<code>Visibility_Logic__c</code>	Long Text	JSON condition to show/hide this individual field. See Section 5.1

Targeting (Which Object Does This Field Save To?)

By default, every field in a section saves to that section's `Object_API_Name__c`. These fields allow per-field overrides:

Field Label	API Name	Type	Description
Target Object API Name	Target_Object_API_Name__c	Text	Overrides the section's object for this specific field. Use when a single section contains fields from two different objects
Target Object Parent Field	Target_Object_Parent_Field__c	Text	If Target_Object_API_Name__c is set to a child object, this is the lookup field on that child pointing to the parent

Lookup Field Configuration

Field Label	API Name	Type	Description
Lookup Target Object	<code>Lookup_Target_Object__c</code>	Text	Required for Lookup fields. API name of the object being searched (e.g., <code>Account</code> , <code>User</code>)
Lookup Search Field	<code>Lookup_Search_Field__c</code>	Text	Comma-separated field API names to search against and display. First field is the primary label; subsequent fields appear as metadata. Example: <code>Name, Email, Phone</code>
Prepopulate	<code>Prepopulate__c</code>	Checkbox	If <code>true</code> , the framework attempts to auto-populate this field with the current page's <code>recordId</code> when in create mode
Key Prefix	<code>Key_Prefix__c</code>	Text (3)	Security lock for prepopulation. The first 3 characters of the expected record ID type (e.g., <code>001</code> for Account, <code>003</code> for Contact, <code>500</code> for Case). Pre-population only fires if the page's <code>recordId</code> starts with this prefix. Prevents a Contact ID from being placed into an Account lookup
Source Field API Name	<code>Source_Field_API_Name__c</code>	Text	When prepopulating, instead of using the <code>recordId</code> itself, copy the value from this field on the source record. Example: set to <code>OwnerId</code> to copy the Account's owner into a User lookup field
Controlling Lookup	<code>Controlling_Lookup__c</code>	Text	The API name of another Lookup field on the same form whose value controls this field. When the controlling lookup changes, this field's value is reactively cleared and optionally re-populated if <code>Source_Field_API_Name__c</code> is configured

Picklist Field Configuration

Field Label	API Name	Type	Description
Override Picklist Values	<code>Override_Picklist_Values__c</code>	Long Text	Comma-separated list to completely replace the Salesforce picklist values. Example: <code>Hardware, Software</code> . Also used for File Upload fields to specify accepted formats (e.g., <code>.pdf, .png, .jpg</code>)
Controller Field	<code>Controller_Field__c</code>	Text	For dependent picklists: the API name of the controlling field. The framework uses Salesforce's schema dependencies by default; pair with <code>Override_Dependency_JSON__c</code> to override
Override Dependency JSON	<code>Override_Dependency_JSON__c</code>	Long Text	JSON to manually define dependent picklist mappings when standard Salesforce dependencies aren't sufficient. See example in Section 5

Formula & Dynamic Query Fields

Field Label	API Name	Type	Description
Formula Logic	<code>Formula_Logic__c</code>	Long Text	JavaScript expression using <code>{FieldAPIName}</code> merge syntax. Evaluated on the client whenever any dependent field changes. Result is stored in memory. Field must be <code>Read_Only__c = true</code> . See Section 5.3
Dynamic SOQL	<code>Dynamic_SOQL__c</code>	Long Text	SOQL query using <code>'{FieldAPIName}'</code> merge syntax as bind variables. Fires when any referenced field changes. Result populates this field's value. See Section 5.4

5. JSON Configuration Reference

5.1 Visibility Logic

Used in both `Form_Section__mdt.Visibility_Logic__c` and `Form_Field__mdt.Visibility_Logic__c`.

Evaluated entirely on the client — no server call is made on each field change.

Operators

Operator	Meaning
<code>equals</code>	Value strictly matches (type-sensitive: <code>true</code> $\neq$ <code>"true"</code> for Checkboxes)
<code>not_equals</code>	Value does not match
<code>includes</code>	Field value contains the target string (useful for multi-select picklist semicolon strings)
<code>excludes</code>	Field value does not contain the target string

Single Condition

```
json
{
  "when": "Is_VIP__c",
  "operator": "equals",
  "value": true
}
```

Important: For Checkbox fields, use boolean `true`/`false` (not strings). For Picklist/Text fields, use string values.

AND Condition

```
json
{
  "operator": "AND",
  "conditions": [
    { "when": "Is_VIP__c", "operator": "equals", "value": true },
    { "when": "Subject", "operator": "not_equals", "value": "" }
  ]
}
```

OR Condition

```
json
```

```
{
  "operator": "OR",
  "conditions": [
    { "when": "Priority", "operator": "equals", "value": "High" },
    { "when": "Priority", "operator": "equals", "value": "Critical" }
  ]
}
```

Referencing a Matrix Cell Value

Matrix cell values are registered in the global state with the prefix `MATRIX__`. You can reference them in any visibility or required logic:

```
json
{
  "when": "MATRIX__Country_Service_Mapping__Domestic_High_Value__US",
  "operator": "equals",
  "value": true
}
```

Key format: `MATRIX__<SectionDeveloperName>__<RowKey>__<ColKey>`

- `SectionDeveloperName` — the `DeveloperName` of the Matrix `Form_Section__mdt` record
- `RowKey` — the `key` value of the row in `Section_Config_JSON__c`
- `ColKey` — the `key` value of the column in `Section_Config_JSON__c`

How Field Values Are Resolved

The `checkLogic` function first looks for the field in the **current row's data context**, then falls back to a **global search** across all rows in all sections (`getGlobalValue()`). This means you can reference fields from entirely different sections in any visibility condition.

5.2 Required Logic

Identical JSON format to Visibility Logic. Placed in `Form_Field__mdt.Required_Logic__c`.

When the condition evaluates to `true`, the field becomes required **in addition to** (not instead of) its `Required__c` static setting.

```
json
```

```
{
  "operator": "AND",
  "conditions": [
    { "when": "Is_VIP__c", "operator": "equals", "value": true }
  ]
}
```

A field with `Required__c = false` and a `Required_Logic__c` condition will only be required when the condition is met. Fields hidden by `Visibility_Logic__c` are **never required**, even if `Required__c = true`.

5.3 Formula Logic

Placed in `Form_Field__mdt.Formula_Logic__c`. The field must have `Read_Only__c = true` and `Save_To_Database__c = true` (unless it is a purely virtual display field).

Uses standard JavaScript expression syntax. Reference other fields using `{FieldAPIName}` merge tags.

Simple Arithmetic

```
{Quantity__c} * {Unit_Price__c}
```

Conditional Expression

```
{Cleanliness_Score} + {Speed_Score} / 2 > 80 ? 'Pass' : 'Fail'
```

Cross-Section References

The formula engine calls `getGlobalValue()`, which searches all rows across all sections. This means you can reference a field from a completely different section:

```
{Section_A_Field__c} + {Section_B_Field__c} * 100
```

Behaviour Notes

- Formulas are recalculated on every field change in the form
- If a referenced field is empty or non-numeric, it is substituted with `@`
- Results are stored in the background `sectionData` state — the UI is updated via the `evaluateVisibility()` repaint cycle to prevent infinite rendering loops
- Fields with `Formula_Logic__c` are automatically disabled in the UI regardless of `Read_Only__c`

5.4 Dynamic SOQL

Placed in `Form_Field__mdt.Dynamic_SOQL__c`. Fires a server-side Apex query in real time when any referenced field value changes.

Syntax

Reference field values using `'{FieldAPIName}'` (with quotes). These are translated by the framework into safe Apex bind variables before execution — **SOQL injection is not possible**.

```
sql

SELECT IsViolated FROM CaseMilestone
WHERE CaseId = '{WhatId}'
AND MilestoneType.Name = 'Resolution'
LIMIT 1
```

```
sql

SELECT OwnerId FROM Case WHERE Id = '{ParentId__c}' LIMIT 1
```

Behaviour Notes

- `LIMIT 1` is automatically appended if not present in the query
- The engine extracts the **first non-Id field value** from the returned record and populates the target field
- If any bind variable is empty (the user hasn't selected a value yet), the query is **skipped** and the field is cleared
- Results are cached per query key — a late-arriving response for an outdated input is discarded (race-condition-safe)
- Errors in the query are silently suppressed on the client to prevent disrupting the end user's form session; check `System.debug` logs for diagnostics
- These fields should have `Save_To_Database__c` set based on whether the result should be persisted

Example: Auto-populate SLA Violation from CaseMilestone

Field: `SLA_Violated__c` (Checkbox on Case)

```
sql
```

```
SELECT IsViolated FROM CaseMilestone
WHERE CaseId = '{Id}'
AND MilestoneType.Name = 'First Response'
LIMIT 1
```

The `{Id}` reference resolves to the current record's Id when in edit mode (populated via `getGlobalValue()`).

5.5 Button Config JSON

Placed in `Form_Config__mdt.Button_Config_JSON__c`. All keys are optional — only specify the values you want to override.

```
json
{
  "cancel":          "Discard",
  "next":            "Continue →",
  "previous":        "← Back",
  "finish":          "Done",
  "submit":          "Submit Request",
  "save":            "Save Changes",
  "saveAndUpload":   "Save & Continue to Upload",
  "saveAndFinish":   "Save & Finish",
  "saveAndAttach":   "Save & Upload Files",
  "msgRecordSaved":   "Your request has been submitted!",
  "msgRecordSavedUpload": "Record saved! Please upload your supporting files.",
  "msgUploadSuccess": "file(s) uploaded successfully."
}
```

Key	When Shown	Default
<code>cancel</code>	Always	<code>Cancel</code>
<code>next</code>	Wizard mode, not last step	<code>Next</code>
<code>previous</code>	Wizard mode, not first step	<code>Previous</code>
<code>finish</code>	After save when upload sections exist	<code>Finish</code>
<code>submit</code>	Submit button before first save	<code>Submit Form</code>
<code>save</code>	Submit button after a record has been saved (edit mode / re-save)	<code>Save Changes</code>
<code>saveAndUpload</code>	When required File Upload fields are present and record not yet saved	<code>Save & Continue to Upload</code>
<code>saveAndFinish</code>	When optional File Upload fields are present (no-upload path)	<code>Save & Finish</code>
<code>saveAndAttach</code>	When optional File Upload fields are present (upload path)	<code>Save & Attach Files</code>
<code>msgRecordSaved</code>	Toast on successful save (no upload required)	<code>Record saved successfully!</code>
<code>msgRecordSavedUpload</code>	Toast on successful save when upload step follows	<code>Record saved! Please upload your files.</code>
<code>msgUploadSuccess</code>	Toast after file upload completes	<code>file(s) uploaded successfully.</code>

5.6 Matrix Section Config JSON

Placed in `Form_Section__mdt.Section_Config_JSON__c`. Only used when `Render_As__c = Matrix`.

Full Structure

json

```

{
  "columns": [
    { "label": "United Kingdom", "key": "GB", "type": "checkbox" },
    { "label": "United States", "key": "US", "type": "checkbox" },
    { "label": "Score", "key": "Score", "type": "number" }
  ],
  "rows": [
    {
      "label": "Domestic High-Value Payment",
      "key": "Domestic_High_Value",
      "cells": {
        "GB": {
          "readonlyLogic": {
            "when": "Maintenance_Levels__c",
            "operator": "excludes",
            "value": "Level-1"
          }
        },
        "US": {
          "readonly": true
        },
        "Score": {
          "type": "number"
        }
      }
    }
  ]
}

```

Column Properties

Property	Required	Description
label	✔	Column header text
key	✔	Unique identifier. Must use only letters, numbers, and underscores. This key is used as part of the DOM ID and the MATRIX__ visibility reference. Spaces or special characters will break accessibility bindings
type	—	checkbox (default), number, or text

Row Properties

Property	Required	Description
<code>label</code>	✓	Row header text shown in the first column
<code>key</code>	✓	Unique identifier. Same naming rules as column keys
<code>cells</code>	—	Object keyed by column key, with per-cell override rules

Cell Override Properties

Property	Description
<code>type</code>	Override the column's default type for this specific cell
<code>readonly</code>	<code>true</code> to statically disable this cell
<code>readonlyLogic</code>	Same JSON structure as <code>Visibility Logic</code> . When the condition is <code>true</code> , the cell is disabled and its value is wiped (garbage collection on disable). Single-condition format only (no AND/OR nesting)

Data Storage

Each matrix cell is persisted as one record in the `Matrix_Data__c` junction object with these fields:

Field	Value
<code>Section_Key__c</code>	<code>DeveloperName</code> of the <code>Form_Section__mdt</code> record
<code>Row_Key__c</code>	The row's <code>key</code> from the JSON config
<code>Column_Key__c</code>	The column's <code>key</code> from the JSON config
<code>Value__c</code>	Cell value as a string (<code>"true"</code> , <code>"false"</code> , or numeric/text string)

Garbage collection: When a cell that was previously saved is disabled by `readonlyLogic`, the framework adds its `Matrix_Data__c` record ID to `_recordsToDelete`, which triggers a hard `Database.delete()` during the next save.

Override Dependency JSON Example (for Picklists)

```
json
```

```
{
  "Hardware": [
    { "label": "Laptop", "value": "Laptop" },
    { "label": "Monitor", "value": "Monitor" }
  ],
  "Software": [
    { "label": "Office 365", "value": "Office 365" },
    { "label": "Salesforce", "value": "Salesforce" }
  ]
}
```

6. LWC Component Reference

6.1 `dynamicRecordForm` — Core Engine

This is the only component that contains form logic. All wrapper components simply pass properties to it.

Public API (`@api` Properties)

Property	Type	Default	Description
<code>formDeveloperName</code>	String	—	Required. The <code>DeveloperName</code> of the <code>Form_Config__mdt</code> record to load
<code>formMode</code>	String	<code>'auto'</code>	Controls data loading behaviour. <code>'auto'</code> = loads data if <code>recordId</code> looks like an existing record (≥ 15 chars); <code>'create'</code> = never loads existing data (ID is treated as context for pre-population only); <code>'edit'</code> = always loads existing data from the ID
<code>recordId</code>	String	—	The Salesforce record ID. In create mode: used for pre-population context. In edit mode: the record to load and edit
<code>objectApiName</code>	String	—	The API name of the current record's object. Used for contextual pre-population in some scenarios
<code>recordTypeId</code>	String	<code>''</code>	Optional. Forces a specific Record Type on save and enables Record Type-aware picklist filtering
<code>isLightningOut</code>	Boolean	<code>false</code>	Set to <code>true</code> when embedding in a Visualforce page via Lightning Out. Switches file upload from <code>lightning-file-upload</code> to a base64-based <code>lightning-input type="file"</code>

Events Fired

Event Name	When	<code>event.detail</code> Shape	Description
<code>close</code>	Cancel, successful save, or Finish	<code>{ recordId: String null }</code>	Signals the parent wrapper to close the modal/tab. <code>recordId</code> is the saved/updated record's ID, or <code>null</code> on cancel
<code>notification</code>	Any toast message	<code>{ title, message, variant }</code>	Bubbles a toast up through the Shadow DOM. Required for Lightning Out / VF page integration where <code>ShowToastEvent</code> doesn't reach the VF container

Both `close` and `notification` fire with `bubbles: true, composed: true` so they cross Shadow DOM boundaries.

6.2 Wrapper Components

These are thin wrappers. They exist to provide the correct default configuration for each use case.

caseCreateWrapper

```
html

<c-dynamic-record-form
  record-id={recordId}
  object-api-name={objectApiName}
  record-type-id={recordTypeId}
  form-developer-name="Case_Intake"
  form-mode="create"
  is-lightning-out={isLightningOut}
  onclose={handleClose}>
</c-dynamic-record-form>
```

@api Property	Type	Notes
recordId	String	Context record ID (e.g., Account ID when creating from Account)
objectApiName	String	Object of the context record
recordTypeId	String	Passed through to dynamicRecordForm
isLightningOut	Boolean	Set to <code>true</code> by the VF page

The `handleClose` method fires a `close` CustomEvent with `bubbles: true, composed: true` and also calls `NavigationMixin.Navigate` (for Lightning standard nav) and `CloseActionScreenEvent` (to close the Quick Action modal).

caseEditWrapper

```
html

<c-dynamic-record-form
  record-id={recordId}
  object-api-name={objectApiName}
  form-developer-name="Case_Intake"
  form-mode="edit"
  onclose={handleClose}>
</c-dynamic-record-form>
```

Close handler simply dispatches `CloseActionScreenEvent`.

taskCreateWrapper

html

```
<c-dynamic-record-form
  record-id={recordId}
  object-api-name={objectApiName}
  form-developer-name="Task_Intake"
  form-mode="create"
  onclose={handleClose}>
</c-dynamic-record-form>
```

calculateQcScore

html

```
<c-dynamic-record-form
  record-id={recordId}
  object-api-name={objectApiName}
  form-developer-name="Case_Quality_Score"
  form-mode="edit"
  onclose={handleClose}>
</c-dynamic-record-form>
```

7. Apex API Reference

DynamicFormController

All `@AuraEnabled` methods are available to any LWC via `import ... from '@salesforce/apex/DynamicFormController.*'`.

getFormMetadata

apex

```
@AuraEnabled(cacheable=true)
public static FormWrapper getFormMetadata(String formDeveloperName, String recordType
```

Fetches all configuration metadata for the form. Marked `cacheable=true` — results are stored in the Lightning Data Service cache. Metadata changes in production require a cache-busting deploy or a browser hard-refresh for users to see them immediately.

Parameter	Description
<code>formDeveloperName</code>	<code>DeveloperName</code> of the <code>Form_Config__mdt</code> record
<code>recordTypeId</code>	Optional. If provided, enables Record Type-aware picklist filtering via <code>ConnectApi.RecordUi.getPicklistValuesByRecordType()</code>

Returns: `FormWrapper` containing the full form configuration with sections, fields, picklist options, and dependency maps.

Throws: `AuraHandledException` if the form is inactive (`Active__c = false`).

`getExistingRecordData`

```
apex
@AuraEnabled
public static Map<String, Object> getExistingRecordData(
    String recordId,
    String objectApiName,
    String queryConfigJson
)
```

Dynamically builds and executes SOQL queries to fetch parent and child records for edit mode. The LWC generates the `queryConfigJson` automatically based on the form metadata — you do not need to call this manually.

Returns: A `Map` with two keys:

- `parent`: `Map<String, Object>` of the parent record's field values. Lookup fields include a `{FieldApiName}_Label` entry with the related record's display name
- `children`: `Map<String, List<Map<String, Object>>>` keyed by child object API name

`saveMultiObject`

```
apex
```

```
@AuraEnabled
public static Map<String, Object> saveMultiObject(
    String parentObjectApiName,
    Map<String, Object> payload,
    Map<String, Object> relationshipMap,
    List<String> recordsToDelete,
    Boolean saveWithoutSharing
)
```

The transactional save engine. Performs all DML within a `Database.setSavepoint()` block.

Parameter	Description
<code>parentObjectApiName</code>	API name of the primary/parent object (e.g., <code>Case</code>)
<code>payload</code>	Hierarchical map of <code>{ objectApiName: {fieldMap} }</code> or <code>{ objectApiName: [{fieldMap}, ...] }</code>
<code>relationshipMap</code>	Map of <code>{ childObjectApiName: lookupFieldOnChild }</code> for linking children to the parent
<code>recordsToDelete</code>	List of record IDs to hard-delete before saving (removed rows and disabled matrix cells)
<code>saveWithoutSharing</code>	If <code>true</code> , uses the <code>SystemModeDML</code> inner class to bypass sharing rules

Returns: `Map` with:

- `parentId`: The Salesforce ID of the upserted parent record
- `childIds`: `Map<String, String>` of `{ childObjectApiName: childRecordId }` for single-record child saves

FLS enforcement: Before setting any field value, the engine checks `f.getDescribe().isCreateable()` or `isUpdateable()`. Fields the user lacks permission for are silently skipped.

searchRecords

apex

```
@AuraEnabled(cacheable=true)
public static List<Map<String, String>> searchRecords(
    String searchTerm,
    String objectApiName,
    String searchFields
)
```

Powers the custom Lookup typeahead. Returns up to 10 matching records.

Parameter	Description
<code>searchTerm</code>	The text the user has typed
<code>objectApiName</code>	Target object to search
<code>searchFields</code>	Comma-separated field API names. All fields are searched with <code>LIKE '%searchTerm%'</code> using OR conditions. First field becomes the display label

Returns: List of maps with keys: `id`, `label`, `meta` (second field value), `details` (all secondary fields formatted as `fieldName: value` lines)

`getRecordDetails`

```
apex

@AuraEnabled(cacheable=true)
public static LookupRecordDetails getRecordDetails(
    String recordId,
    String objectApiName,
    String searchFields
)
```

Fetches the display label and details for a known record ID. Used to populate lookup field labels in edit mode.

`getSourceRecordData`

```
apex
```

```
@AuraEnabled
public static Map<String, Map<String, Object>> getSourceRecordData(
    String sourceRecordId,
    List<String> sourceFields
)
```

Fetches field values from the context record to copy into the form on create. Used by the `Source_Field_API_Name__c` pre-population mechanism.

Returns: Map of `{ sourceFieldApiName: { value: rawValue, label: displayLabel } }`. For lookup fields, `label` contains the related record's Name.

`executeDynamicQuery`

```
apex

@AuraEnabled
public static SObject executeDynamicQuery(
    String soqlQuery,
    Map<String, Object> bindParams
)
```

Executes a single-row SOQL query using `Database.queryWithBinds()` with `AccessLevel.SYSTEM_MODE`. The LWC translates `{FieldName}` merge syntax into Apex bind variables before calling this method.

Returns the first matching `SObject`, or `null` if no results. Errors are silently suppressed and `null` is returned to avoid disrupting the user's form session.

`uploadFile`

```
apex

@AuraEnabled
public static Id uploadFile(String parentId, String fileName, String base64Data)
```

Creates a `ContentVersion` record linked to `parentId`. Used exclusively in the Lightning Out / Salesforce context (max 4MB per file enforced on the client).

`deleteRecord`

```
apex
```

@AuraEnabled

```
public static void deleteRecord(String recordId)
```

Generic single-record delete. Used for: file removal, record rollback on cancel, and child record deletion.

PicklistDependencyHelper

apex

```
public static Map<String, List<Map<String, String>>> getSerializedDependencyMap(  
    Schema.SObjectField dependentField  
)
```

Decodes Salesforce's internal `validFor` base64 bitset from `PicklistEntry` records to determine which dependent values are valid for each controlling value. This is a reflection-based approach that works around the lack of a native dependency API.

The result is a map: `{ controllingValue: [ {label, value}, ... ] }`.

8. Admin Configuration Guide

This section walks through creating a new form end-to-end using only clicks in Setup.

Step 1: Plan Your Form

Before touching metadata, answer these questions:

1. What is the **primary object** this form creates or edits? (e.g., `Case`)
 2. Are there **child objects** that should be created inline? (e.g., `Work_Part__c`)
 3. Which fields need **conditional visibility**? (e.g., "Show Description only if Is VIP is checked")
 4. Will users access this form from a **Quick Action**, a **VF page**, or a **Lightning Tab**?
 5. Are there **file uploads** required?
 6. Is a **Record Type** involved?
-

Step 2: Create the Form Config Record

Navigate to: **Setup** → **Custom Metadata Types** → **Form Config** → **Manage Records** → **New**

Field	What to Enter
Label	Human-readable name (e.g., <code>Maintenance Request</code>)
Developer Name	Unique identifier with underscores (e.g., <code>Maintenance_Request</code>). This is what you pass to the LWC
Active	☒ Checked
Object API Name	Your primary object (e.g., <code>Case</code>)
Form Title	The text shown on the form card (e.g., <code>New Maintenance Request</code>)
Form Icon	SLDS icon (e.g., <code>standard:case</code>). See SLDS Icons for options
Display Mode	<code>Single Page</code> for most forms; <code>Wizard</code> for multi-step flows
Form Instructions	Optional banner HTML (e.g., <code>For maintenance requests only</code>)
Button Config JSON	Optional — leave blank to use defaults
Save Without Sharing	Check if the form creates records that are immediately reassigned by Assignment Rules

Step 3: Create Form Section Records

Navigate to: **Setup** → **Custom Metadata Types** → **Form Section** → **Manage Records** → **New**

Create one section record per logical grouping of fields.

Example: General Info section (parent object)

Field	Value
Label	General Info
Developer Name	General_Info
Form Config	Maintenance_Request (select your config)
Object API Name	Case
Order	1
Number of Columns	2
Show On	Both
Collapse by Default	Unchecked (open by default)

Example: Work Parts section (child object)

Field	Value
Label	Work Parts
Developer Name	Work_Parts
Form Config	Maintenance_Request
Object API Name	Work_Part__c
Relationship Parent Field	Maintenance_Request__c (the lookup on Work_Part__c to Case)
Order	2
Allow Multiple Rows	☒ Checked
Collapse by Default	☒ Checked

Step 4: Create Form Field Records

Navigate to: **Setup → Custom Metadata Types → Form Field → Manage Records → New**

Create one record per field you want on the form.

Example: Subject field

Field	Value
Label	Subject
Developer Name	Subject
Form Section	General_Info (select your section)
Field API Name	Subject
Field Type	Text
Order	1
Required	☒ Checked
Save To Database	☒ Checked

Example: Account lookup with pre-population

Field	Value
Label	Account
Developer Name	Account
Form Section	General_Info
Field API Name	AccountId
Field Type	Lookup
Lookup Target Object	Account
Lookup Search Field	Name, AccountNumber
Prepopulate	☒ Checked
Key Prefix	001
Order	2
Required	☒ Checked
Save To Database	☒ Checked

Example: Priority picklist with default value

Field	Value
Label	Priority
Developer Name	Case_Priority
Form Section	General_Info
Field API Name	Priority
Field Type	Picklist
Default Value	Medium
Order	3
Required	☒ Checked
Save To Database	☒ Checked

Step 5: Add Visibility Logic (Optional)

To show a field only when another field has a specific value, edit the field's **Visibility Logic** field.

Example: Show Description only when Is VIP is checked AND Subject is not empty

In `Form_Field__mdt.Description.Visibility_Logic__c`:

```
json
{
  "operator": "AND",
  "conditions": [
    { "when": "Is_VIP__c", "operator": "equals", "value": true },
    { "when": "Subject", "operator": "not_equals", "value": "" }
  ]
}
```

Step 6: Wire Up Deployment

Choose your deployment method and follow [Section 10](#).

Step 7: Test

1. Navigate to the trigger point (Quick Action, VF page, etc.)
 2. Verify all fields render correctly
 3. Test conditional visibility by changing controlling fields
 4. Test required validation by attempting to submit empty required fields
 5. Test save — verify parent and all child records are created correctly
 6. Test edit mode — verify existing values load correctly
-

9. Developer Guide

9.1 The `sectionData` State Model

The framework manages all form state in a flat map called `sectionData`, keyed by UUID:

```
javascript

sectionData = {
  "uuid-for-row-1": {
    "Subject":    "Fix cooling unit",
    "Priority":    "High",
    "AccountId":  "001XXXXXXXXXXXXX",
    "Is_VIP__c":  true
  },
  "uuid-for-work-part-1": {
    "Equipment__c": "01tXXXXXXXXXXXXX",
    "Quantity__c":  2
  }
}
```

Each section row (including the single default row of non-repeating sections) gets a UUID. Child row UUIDs are tracked for deletion when a row is removed.

The `sections` array in the component state mirrors the metadata structure but adds runtime state (visibility, current values, lookup options, etc.). **Do not mutate `sections` directly** from outside the component — use the `@api` properties.

9.2 The Reactive Pipeline

Every field change triggers this sequence:

```

handleFieldChange()
  └─ Update sectionData[rowId][fieldApi] = newValue
  └─ filterDependencies()      - update dependent picklist options
  └─ calculateFormulas()      - recalculate all formula fields (background state
only)
  └─ evaluateVisibility()      - sync UI values, apply show/hide, apply required
  └─ applyMatrixRules()        - apply readonlyLogic to matrix cells
  └─ evaluateDynamicQueries() - fire Apex queries for Dynamic SOQL fields

```

The cycle always ends in `evaluateVisibility()`, which is the single source of truth for pushing background state changes to the rendered UI. This design prevents infinite re-render loops.

9.3 File Upload Behaviour

The framework uses two different upload mechanisms based on context:

Context	<code>isLightningOut</code>	Component Used	Max File Size
Quick Action / Lightning Page	<code>false</code>	<code>lightning-file-upload</code>	Standard Salesforce limit
Visualforce / Lightning Out	<code>true</code>	<code>lightning-input</code> <code>type="file"</code> + base64 upload via Apex	4 MB per file (enforced client- side)

For Lightning Out, after each file is selected, the framework:

1. Reads the file using `FileReader.readAsDataURL()`
2. Strips the base64 prefix
3. Calls `DynamicFormController.uploadFile()` which creates a `ContentVersion` linked to the saved record

Note: File Upload fields are **hidden** until the record has been saved (i.e., after the first submit creates the parent record ID). This is by design — files must have a parent record to link to.

9.4 Adding a New Field Type

1. Add the new value to `Form_Field__mdt.Field_Type__c` picklist
2. In `dynamicRecordForm.html`, add a new `<template if:true={field.isYourNewType}>` block with the appropriate Lightning component

3. In `dynamicRecordForm.js`, in the `initializeFields()` method, add the boolean flag:
`isYourNewType: Boolean(typeLower === 'your_new_type')`
 4. Ensure `isStandard` in `initializeFields()` excludes your new type
 5. Handle the field value in `handleFieldChange()` if it has non-standard change event behaviour
-

9.5 Custom Component Injection (Advanced)

If a section has `Custom_Component_Name__c` populated, the form renders a custom child LWC for that section instead of a standard field grid.

Contract: Your custom component must implement:

- Accept `@api recordId` for context
- Accept `@api isLoading` to disable its UI during parent form saves
- Expose `@api getCustomData()` — returns an object compatible with the framework's payload format
- Fire a `change` event if it mutates state that should trigger the parent's visibility evaluation

The framework does not automatically call `getCustomData()` — you must add the call manually in `handleSubmit()` in `dynamicRecordForm.js` after the standard payload is built.

9.6 Pre-population Flow (Detailed)

On **create mode**, the framework evaluates pre-population in this priority order:

1. **Source Field copy** (`Source_Field_API_Name__c` is set + `Key_Prefix__c` matches):
Fetches the specified field from the context record and copies its value to this field.
Example: Copy `OwnerId` from the Account into the Case's `Account_Manager__c` lookup.
 2. **Direct ID pre-population** (`Prepopulate__c = true` + `Key_Prefix__c` matches): Sets the field value to the context `recordId` directly. Simultaneously fetches the record's label via `getRecordDetails()` so the lookup displays a name, not an ID.
 3. **Current User pre-population** (`Prepopulate__c = true` + `Lookup_Target_Object__c = User` + no `Key_Prefix__c`): Auto-sets the field to the logged-in user's ID using the metadata response's `currentUserId`/`currentUserName` — **no additional Apex call needed**.
 4. **Default Value** (`Default_Value__c` is set): Applied as the initial value on create. Supports: string values for text/picklist, `true`/`false` for checkboxes, numeric strings for numbers.
-

10. Deployment & Integration Guide

10.1 Quick Actions (Most Common)

Quick Actions are the standard way to surface the form on a record page.

Steps:

1. Navigate to **Setup** → **Object Manager** → **[Your Object]** → **Buttons, Links, and Actions** → **New Action**
2. Action Type: `Lightning Web Component`
3. Lightning Web Component: Select your wrapper (e.g., `caseCreateWrapper`)
4. Label and save
5. Add the action to the object's **Page Layout** or directly to the **Lightning Page** via the Highlights Panel component's action configuration

Behaviour:

- The action opens as a modal overlay
 - `recordId` and `objectApiName` are automatically passed by Salesforce
 - On save or cancel, `CloseActionScreenEvent` closes the modal
 - On save, `NavigationMixin.Navigate` navigates to the newly created record
-

10.2 Lightning Record Pages

To embed the form directly on a record page (not as a modal):

1. Edit the Lightning Page in **App Builder**
 2. Drag the `dynamicRecordForm` component onto the page
 3. In the component properties panel, set:
 - **Form Configuration Name:** Your `DeveloperName` (e.g., `Case_Intake`)
 - **Form Mode:** `edit` for record pages
 - **Record Type ID:** Optional — leave blank to auto-detect from the current record
-

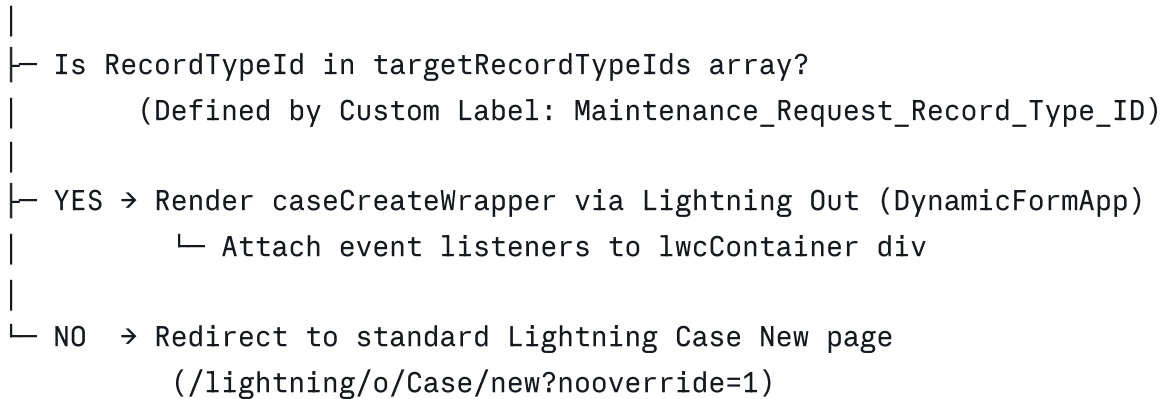
10.3 Visualforce Page Integration (Legacy Console)

The `caseCreatePage` Visualforce page provides **routing** for the standard "New Case" button override. It inspects the Record Type and routes accordingly:

New Case button clicked



caseCreatePage.page



Setup steps:

1. The Custom Label `Maintenance_Request_Record_Type_ID` must contain the 15-character Record Type ID for Maintenance requests
2. The `NewCase` action override on the Case object must point to `caseCreatePage`
3. This only runs on the **standard Classic/Console** new case path — the Lightning Quick Action path does not go through VF

Console App Event Handling (VF):

The VF page attaches listeners to the `lwcContainer` div that:

- `notification` event → fires `sforce.one.showToast()`
- `close` event → navigates to the new record and closes the tab using `sforce.console.closeTab()`

10.4 Aura Wrapper (Custom Tabs / Console Apps)

The `MaintenanceFormAuraWrapper` Aura component is used when you need to open the form in a **custom Console tab** (not a Quick Action modal):

URL to open tab:
`/lightning/cmp/c__MaintenanceFormAuraWrapper?`
`c__recordId=001XXX&c__recordTypeId=012XXX`

The Aura component reads `c__recordId` and `c__recordTypeId` from the URL parameters and passes them to `caseCreateWrapper`.

Console tab close behaviour:

- Uses `lightning:workspaceAPI.getFocusedTabInfo()` then `closeTab()`
 - In a Standard (non-Console) app: the LWC handles navigation itself via `NavigationMixin` — the Aura component does nothing extra
-

11. Real-World Form Examples

11.1 Case Intake Form (`Case_Intake`)

Attribute	Value
Primary Object	<code>Case</code>
Display Mode	Single Page
Sections	General Info, Picklists, Date & Date Time, Case Comments (child), Work Parts (child), Upload Supporting Files, Country Service Mapping (Matrix), Other Details
Record Types	Maintenance, Service
Notable Features	Pre-population of Account, Contact, Parent Case; dependent picklists (Reason → Sub-Reason, Category → Sub-Category); Matrix section with per-cell readonly logic; conditional visibility on Description and Priority; CaseComment child records; Work Part child records

Section order and visibility summary:

#	Section	Object	Allow Multiple	Visible When
1	General Information	Case	No	Always
2	Picklists	Case	No	Always
3	Date & Date Time	Case	No	Collapsed by default
4	Case Comments	CaseComment	Yes	Is_VIP__c=true AND Priority=High
5	Work Parts	Work_Part__c	No	Collapsed by default
6	Upload Supporting Files	Case	No	After save only
7	Country Service Mapping	Matrix_Data__c	No (Matrix)	Is_VIP__c=true AND Priority=Medium
8	Other Details	Case	No	Always

11.2 Task Intake Form (Task_Intake)

Attribute	Value
Primary Object	Task
Display Mode	Single Page
Sections	Task Details
Notable Features	Pre-populates WhatId (Case lookup) from the Case context (Key_Prefix__c = 500); pre-populates OwnerId (User) from current user; Override Picklist Values for Subject

Fields:

Field	APIName	Notes
Subject	Subject	Picklist with overridden values: Service Request, Breakdown, Maintenance
Status	Status	Required
Owner	OwnerId	Pre-populated with current user
Due Date	ActivityDate	Date field
Priority	Priority	Optional
Case	WhatId	Lookup to Case; pre-populated from context; read-only
Comments	Description	Long Text Area

11.3 QC Score Form (Case_Quality_Score)

Attribute	Value
Primary Object	Case
Form Mode	Edit only (Show_On__c = Edit) on the section)
Display Mode	Single Page
Notable Features	Formula field (Final_Audit_Grade__c) auto-calculates Pass/Fail from numeric score inputs

Formula example:

Final_Audit_Grade__c formula:

```
({Cleanliness_Score} + {Speed_Score}) / 2 > 80 ? 'Pass' : 'Fail'
```

Cleanliness_Score and Speed_Score have Save_To_Database__c = false — they exist only as in-memory values to drive the formula. Only Final_Audit_Grade__c (the result) is saved to the database.

12. Security Model

Field Level Security (FLS)

The `createSObject()` private method in `DynamicFormController` explicitly checks permissions before setting any field value:

```
apex

if (f.getDescribe().isCreateable() || f.getDescribe().isUpdateable()) {
    // set value
}
```

Fields the running user lacks permission to edit are silently skipped — they will not cause an error, but they will not be saved. This means admins should ensure that all fields configured in metadata are accessible to the profiles/permission sets of users who will use the form.

Sharing Rules

By default, all Apex runs `with sharing`. If `Save_Without_Sharing__c = true` on the Form Config, the `SystemModeDML` inner class is used for DML operations:

```
apex

@TestVisible
private without sharing class SystemModeDML {
    public void doUpsert(SObject record) { upsert record; }
    public void doDelete(List<SObject> records) { Database.delete(records); }
}
```

This class is annotated `@TestVisible` to allow unit test coverage. The outer class remains `with sharing` — only the actual DML operations bypass sharing, not the queries.

SOQL Injection Prevention

- Standard queries: All dynamic SOQL uses `:recordId` bind variables and `String.escapeSingleQuotes()`
- Dynamic SOQL fields: The LWC translates `{FieldName}` merge syntax to Apex bind variables (`:FieldName`) before sending to `executeDynamicQuery()`, which uses `Database.queryWithBinds()` — injection is architecturally impossible

Lightning Out Security

The Visualforce page (`caseCreatePage`) does not expose any additional attack surface beyond what the connected user can already access. The `DynamicFormApp` Aura application (`extends="ltnng:outApp"`) inherits the session's permission context.

13. Performance & Governor Limits

Caching Strategy

Method	Cached?	Notes
<code>getFormMetadata</code>	 <code>cacheable=true</code>	Cached in Lightning Data Service. Near-instant on repeat loads
<code>searchRecords</code>	 <code>cacheable=true</code>	Typeahead results are cached per search term
<code>getRecordDetails</code>	 <code>cacheable=true</code>	Lookup label lookups are cached
<code>getExistingRecordData</code>		Must always be fresh
<code>saveMultiObject</code>		DML cannot be cached
<code>executeDynamicQuery</code>		Must always be fresh

Governor Limit Considerations

Scenario	Mitigation
Many child records in one save	<code>saveMultiObject</code> uses <code>upsert List<SObject></code> — one DML statement per object type regardless of row count
Many picklist fields on one form	<code>ConnectApi.RecordUi.getPicklistValuesByRecordType()</code> called once per form load; result covers all fields
Lookup search on every keystroke	Search only fires when input length ≥ 2 characters; <code>_activeSearchTerms</code> tracking discards stale responses
Dynamic SOQL on every field change	<code>_activeSoqlQueries</code> tracking ensures only the latest query result is applied; stale late responses are discarded
Deep form with many sections	<code>@wire</code> metadata caching means the metadata fetch contributes 0 SOQL on repeat loads within the session

Large Matrix Grids

Matrix sections with many rows $\times$ many columns can generate a large number of

`Matrix_Data__c` records on save. Ensure the `Matrix_Data__c` object's sharing model and governor limits are considered for high-volume usage.

14. Known Limitations & Workarounds

Limitation	Detail	Workaround
Metadata cache on <code>getFormMetadata</code>	After changing <code>Form_Config__mdt</code> , <code>Form_Section__mdt</code> , or <code>Form_Field__mdt</code> records in production, users may see stale form configurations until the browser cache expires	Have users press Ctrl+Cmd+Shift+F to refresh (Ctrl+Cmd+Shift+F) to refresh the browser tab
File upload max 4 MB in VF/Lightning Out	Enforced client-side in <code>handleCustomFileUpload()</code>	Use the standard file upload (not VF) for large files
No native rich-text input support	<code>Field_Type__c</code> does not include a Rich Text editor	Configure as plain text, or use <code>input-rich</code> extending the Section 9.4
Picklist dependency <code>validFor</code> decoding	<code>PicklistDependencyHelper</code> uses base64 decoding of Salesforce's internal schema structure	Low risk — the schema is stable for major changes; the internal fallback is <code>Override__c</code>
Formula fields display as read-only text	Formula results that need to be displayed must use <code>Field_Type__c = Text</code> with <code>Read_Only__c = true</code>	This is the intended workaround
Matrix cell <code>readonlyLogic</code> supports single conditions only	No AND/OR nesting inside <code>readonlyLogic</code> at the cell level	Split complex conditions even at the cell level; use <code>visibility</code> for single <code>include</code> operator against picklist value
ConnectAPI picklist method (Spring '26+)	<code>ConnectApi.RecordUi.getPicklistValuesByRecordType()</code> requires API version 66.0+. Orgs below this version will silently fall back to schema-based picklist values	Ensure <code>DynamicForm</code> uses API version 66.0+
File fields require save before upload	The file upload section is hidden until the record is created	This is by design; the parent record must be saved first

Limitation	Detail	Workaround
		most workflow

15. Troubleshooting

Form displays a loading spinner forever

Likely cause: The @wire call to getFormMetadata failed silently.

Check:

1. Verify Active__c = true on the Form Config record
2. Verify formDeveloperName passed to the component matches exactly the DeveloperName on the Form Config (case-sensitive)
3. Check browser console for Apex errors
4. Check the user's FLS for Form_Config__mdt, Form_Section__mdt, Form_Field__mdt — these custom metadata types must be readable

Fields are not pre-populating

Check:

1. Prepopulate__c = true on the field
2. Key_Prefix__c is set and matches the first 3 characters of the context record's ID
3. Lookup_Target_Object__c is set correctly
4. The form is in create mode (formMode = 'create' or 'auto')
5. If using Source_Field_API_Name__c, verify the field exists and is accessible on the source record type

Dependent picklist is showing all values instead of filtered values

Check:

1. Controller_Field__c on the dependent field is set to the API name of the controlling field
2. If using Override_Dependency_JSON__c: verify the JSON keys exactly match the controlling picklist values (case-sensitive)
3. If using standard Salesforce dependencies: verify the dependency is set up correctly in the object's picklist configuration

4. For Record Type filtering: verify `recordTypeId` is being passed to the component
-

Child records are not being created

Check:

1. `Object_API_Name__c` on the Form Section is set to the **child** object (not the parent)
 2. `Relationship_Parent_Field__c` on the Form Section is the API name of the lookup/master-detail **on the child object**
 3. The child fields have `Save_To_Database__c = true`
 4. At least one non-prepopulated, non-file field in the child section has a value entered (empty child rows are excluded from save)
 5. Check the user has Create permission on the child object
-

Visibility logic is not working

Check:

1. JSON syntax is valid — use a JSON validator
 2. For Checkbox fields: use boolean `true`/`false` (not string `"true"`/`"false"`)
 3. Field API name in `when` is the exact Salesforce API name (case-sensitive)
 4. For multi-select picklist references: use `includes`/`excludes` operators, not `equals`
 5. For cross-section references: the referenced field must exist in the form (in any section)
-

Formula field is showing the wrong value

Check:

1. `Formula_Logic__c` uses `{FieldApiName}` syntax (curly braces, exact API name)
 2. The field has `Read_Only__c = true` (formula fields are auto-disabled, but this is best practice)
 3. Referenced fields are not empty — empty/non-numeric values are substituted with `0`
 4. JavaScript expression syntax is valid — test the expression in browser DevTools console first
-

Dynamic SOQL field is not populating

Check:

1. `{FieldApiName}` syntax is used for bind variables (with single quotes in SOQL: `WHERE Id = '{FieldApiName}'`)
 2. All referenced bind variable fields have non-empty values — the query is skipped if any dependency is empty
 3. The SOQL query returns a result — test it manually in Developer Console
 4. The query returns a field other than `Id` — the engine extracts the first non-Id field value
 5. Check `System.debug` logs in the Developer Console for `Dynamic Query Error:` messages
-

Form shows "Form is currently disabled"

The `Active__c` flag on the Form Config record is unchecked. Enable it in Setup.

Quick Action modal closes immediately without showing the form

Check:

1. The wrapper LWC is correctly specified in the Quick Action definition
 2. The `handleClose` method is not being called prematurely during initialization
 3. Check browser console for JS errors during component mount
-

16. Framework vs. OmniStudio

Use this table when evaluating whether to deploy this framework or Salesforce OmniStudio OmniScript for a given requirement.

Dimension	Dynamic LWC Framework	OmniStudio OmniScript
Licensing	Free — 100% native platform	Requires additional OmniStudio licensing
Configuration UX	CMDT records + JSON (developer/admin)	Visual drag-and-drop canvas (admin-friendly)
Performance	Fast — native LWC, minimal DOM, optimized reactive state	Moderate — large JSON configurations, heavier DOM
Multi-object save	Native — parent + N child objects in one atomic transaction	Complex — requires Integration Procedures or nested Edit Blocks
Client-side reactivity	Instant — no server round-trips for visibility/required changes	Server round-trips for some reactive behaviours
2D Matrix grids	Built-in with per-cell logic	Not natively available
UI customization	Full control — SLDS classes and CSS overrides per field	Rigid — custom styling requires overriding global templates
Maintenance overhead	Single codebase — updating the engine updates all forms	Each OmniScript must be individually versioned and activated
Learning curve	Moderate — requires understanding of JSON config schemas	Low for admins; high for complex scenarios
Version control	Standard SFDX/Git — CMDT deploys like any metadata	Separate versioning system; can drift from source control

Recommendation: Use this framework for complex, multi-object transactional forms with real-time reactivity requirements. Consider OmniStudio for simpler, admin-managed guided flows where visual configuration is a priority and multi-object transactionality is not required.

17. Packaging & Distribution

To distribute this framework to other Salesforce orgs or teams:

Components to Include in the Package

Custom Metadata Types (schema only — not data):

- Form_Config__mdt
- Form_Section__mdt
- Form_Field__mdt

Custom Objects:

- `Matrix_Data__c` (junction object for matrix grids)

Apex Classes:

- `DynamicFormController` (and its test class)
- `PicklistDependencyHelper` (and its test class)

LWC:

- `dynamicRecordForm` (core engine — required)
- Wrapper components are org-specific and typically not packaged

API Version Requirement: `DynamicFormController` must target **API version 66.0 or higher** for `ConnectApi.RecordUi.getPicklistValuesByRecordType()` support.

What Receiving Teams Configure

After installing the package, adopting teams:

1. Create their own `Form_Config__mdt`, `Form_Section__mdt`, and `Form_Field__mdt` records in their org
2. Build wrapper LWC components pointing at their form `DeveloperName` values
3. Deploy wrapper components as Quick Actions or on Lightning Pages

Sample CMDT Deployment

Export sample form configuration as CMDT records using SFDX for an example form:

```
bash

sfdx force:data:tree:export \
  -q "SELECT Label, DeveloperName, Active__c, Object_API_Name__c FROM Form_Config__m"
  -p \
  -d ./sample-data
```

This allows receiving teams to instantly see a working example and understand the JSON structure.

Documentation generated from production codebase. For issues or enhancements, raise a request with the Salesforce Architecture team.